

04 · 4비트 감산기

VS Code 사전 시뮬레이션 → Vivado 2026.1 → 보드 실험

d는 a-b의 하위 4비트이고 bor는 $a < b$ 일 때만 1이다. 같은 수끼리 빼면 borrow는 0이다.

1. 템플릿을 clone하고 이 회로의 workspace를 연다.
2. 예상값을 계산하고 RTL·TB·XDC를 직접 작성해 시뮬레이션한다.
3. Vivado 결과와 실제 보드 동작을 비교한다.
4. 자신의 사진·영상·레포트를 GitHub에 연결한다.

작성일 2026. 09. 10. · 이해리

실습 목차

1부 · VS Code 사전 실습

새 창·workspace·확장·코드 열기

예상값·작업 실행·로그·파형·실험 전 레포트

2부 · Vivado 실습

프로젝트·소스·top·시뮬레이션·핀·비트스트림

3부 · 결과 정리

보드·사진·영상·GitHub·실험 후 레포트

전체 목차 PDF 열기

시뮬레이션 도구 확인

Git·Python 3.10 이상·VS Code·Icarus Verilog를 설치한다. Windows PowerShell에서 한 줄씩 확인한다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

설치·PATH 변경 후 VS Code를 다시 연다. Linux·macOS·WSL은 python 대신 python3를 사용한다.

설치와 빈 템플릿 안내

v2.0.0 템플릿을 새 이름으로 clone

아래는 Windows PowerShell 명령이다. 줄 끝 문자는 다음 줄로 명령을 이어 준다.

```
git clone --branch v2.0.0 `
https://github.com/Glaysia/fpga-lab-template.git `
lab1_04_subtractor4
cd lab1_04_subtractor4
git switch -c main
code LAB1.code-workspace
```

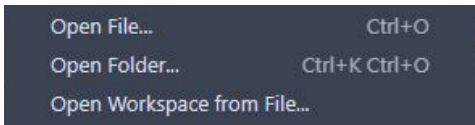
회로마다 마지막 폴더 이름을 바꾼다. 시작 파일은 주석뿐이며 코드 이미지를 보며 직접 작성한다.
고정된 수업 버전 v2.0.0

File → New Window

VS Code 상단 File → New Window. Ctrl+Shift+N도 같은 동작이다.



새 창의 File → Open Workspace from File...을 누른다.



프로젝트 하나의 workspace 열기

1. 방금 clone한 lab1_04_subtractor4 폴더를 연다.
2. LAB1.code-workspace를 선택하고 Open. 모든 실습의 workspace 파일명은 같다.
3. Explorer에 PROJECT 하나와 src·sim·constraints·tools 폴더가 보이는지 확인한다.
4. src/design.v·sim/tb_design.v·constraints/pins.xdc는 아직 주석뿐이다.
5. simulation.json은 실행할 RTL·TB 파일과 TB top을 지정하는 설정이다.

추천 확장 설치

1. 왼쪽 Extensions 아이콘을 누른다.
2. 검색창에 @recommended를 입력한다.
3. slang의 Install: Verilog/SystemVerilog 편집·진단.
4. VaporView의 Install: VCD 파형 확인.
5. vscode-pdf의 Install: 코드 옆에서 PDF 교안 열기.

WSL 사용자는 WSL 창에서 필요할 경우 Install in WSL을 누른다. 확장 설치와 Python·Icarus 설치의 별개다.

src에 RTL 파일 만들기

1. Explorer에서 src 왼쪽 화살표를 눌러 펼친다.
 2. design.v 우클릭 → Rename → sub_4bit.v 입력.
 3. 파일을 두 번 눌러 열고 뒤의 코드 이미지를 보며 전체 코드를 입력한다.
 4. 추가 RTL이 있으면 src 우클릭 → New File로 만든다.
 5. 전체 RTL 파일 목록은 다음 장의 src 표를 따른다.
 6. File → Save All로 모든 파일을 저장한다.
- 설계 모듈명: sub_4bit. 파일명과 module 이름을 구분한다.

sim에 테스트벤치 만들기

1. sim을 펼치고 tb_design.v 우클릭 → Rename.
2. 새 파일명: tb_sub_4bit.sv
3. 뒤의 TB 코드 이미지에 있는 선언·DUT 연결·입력 자극·예상값
검사를 직접 작성한다.
4. wave.vcd를 만드는 dumpfile·dumpvars와 종료 조건까지 작성한다.
5. TB 모듈명: tb_sub_4bit
6. File → Save All. TB를 합성용 RTL 폴더에 넣지 않는다.

작성할 RTL 파일 목록

폴더	파일명
src	sub_4bit.v

각 파일을 New File로 만들고 코드 이미지의 전체 내용을 작성한다.
파일마다 module 선언과 endmodule을 확인한다.

simulation.json을 내 파일에 맞추기

Explorer에서 simulation.json을 두 번 눌러 연다.

sources 배열: 앞의 모든 RTL 파일명 앞에 src/를 붙여 등록한다.

한 파일 예: "sources": ["src/sub_4bit.v"]

testbench: sim/tb_sub_4bit.sv

simulation_top: tb_sub_4bit

sources의 각 경로와 testbench·simulation_top 값은 큰따옴표로 감싼다.
여러 경로는 쉼표로 구분한다.

파일명·확장자·괄호·쉼표를 확인하고 Save All.

입출력과 동작 규칙

입력 $a[3:0]$, $b[3:0]$ / 출력 $d[3:0]$, bor

d 는 $a-b$ 의 하위 4비트이고 bor는 $a < b$ 일 때만 1이다. 같은 수끼리 빼면 borrow는 0이다.

$DIP1-4 = a[3:0]$, $DIP5-8 = b[3:0]$. $LED1 = bor$, $LED2-5 = d[3:0]$.

실행 전에 예상값 작성

입력 또는 조건	예상 결과
0-0	bor=0, d=0
0-1	bor=1, d=15
15-0	bor=0, d=15

10-20ns의 0-1과 170-180ns의 1-1에서 borrow가 다름을 확인한다.
예상값을 계산한 근거를 자신의 말로 설명한다.

RTL 구조를 설명한다

실제 배포 RTL을 VS Code에서 연 화면이다.

설계 top: sub_4bit

```
COMMON > rtl > @ sub_4bit.v >...  
1 module sub_4bit(input wire [3:0] a,b, output wire [3:0] d, output wire bor);  
2     assign d = a - b;  
3     assign bor = a < b; // Equal operands do NOT borrow.  
4 endmodule
```

d는 a-b의 하위 4비트이고 bor는 a<b일 때만 1이다. 같은 수끼리 빼면 borrow는 0이다.

포트 선언·비트 폭·출력 연결을 짚어 설명한다. 저장소 소스를 복사해 사용해도 된다.

배포 소스 확인

테스트벤치와 통과 기준

시뮬레이션 top: `tb_sub_4bit`

256개 입력 조합을 모두 검사한다. 각 자극은 10ns, 종료 시각은 2560ns다.

기대값과 실제 출력이 다르면 `$fatal`로 중단한다. 검사 횟수와 watchdog도 확인한다.

통과 문구: `LAB1_PASS sub_4bit cases=256`

원본 레거시 `testbench.v`와 별도의 자기검사 TB는 파일과 역할을 구분한다.

TB · 입력과 기대값

tb_sub_4bit.sv · 1-12행 · 실제 VS Code 화면

```
1 | timescale 1ns/1ps
2 | module tb_sub_4bit;
3 |     reg [3:0] a,b; wire [3:0] d; wire bor; sub_4bit dut(a,b,d,bor);
4 |     integer n,checked=0;
5 |     reg [4:0] expected;
6 |     initial begin
7 |         $dumpfile("wave.vcd"); $dumpvars(0,tb_sub_4bit);
8 |
9 |         for(n=0;n<256;n=n+1) begin
10 |             {a,b}=n;
11 |             expected=((n/16)<(n%16)) ? 16 : 0 | (((n/16)-(n%16)) & 15);
12 |             #10;
```

상위 비트에는 $a < b$ 인 경우의 borrow를 넣고, 하위 4비트에는 차의 하위 4비트를 넣는다. $a=b$ 도 포함한다.

DUT는 검사할 회로다. dumpfile·dumpvars는 파형 파일에 신호를 남긴다.

TB · 비교와 종료

tb_sub_4bit.sv · 13-22행 · 실제 VS Code 화면

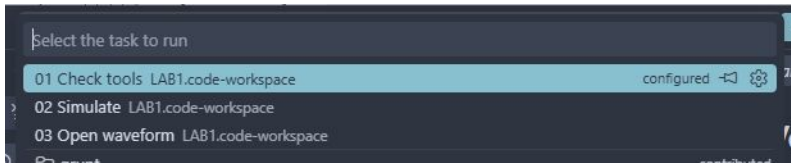
```
13     if ({bor,d} != expected)
14         $fatal(1,"FAIL sub_4bit vector=%0d expected=%h actual=%h",n,expected,{bor,d});
15     checked=checked+1;
16 end
17 if(checked!=256) $fatal(1,"Incomplete test");
18 $display("LAB1_PASS sub_4bit cases=%0d",checked);
19 $finish;
20 end
21 initial begin #100000; $fatal(1,"Watchdog"); end
22 endmodule
```

!=는 X·Z를 포함한 불일치도 잡는다. 다르면 \$fatal로 중단하고 어떤 입력에서 틀렸는지 출력한다.

checked가 전체 검사 수와 같은지 확인한 뒤 PASS와 \$finish. watchdog은 종료되지 않는 테스트를 실패시킨다.

저장 → Terminal → Run Task

File → Save All 다음 Terminal → Run Task...을 누른다.



1. 01 Check tools를 실행하고 도구 버전을 확인한다.
2. 02 Simulate를 선택하고 종료까지 기다린다.
3. 03 Open waveform으로 생성된 VCD를 연다.

작업이 없으면 올바른 .code-workspace를 열었는지 확인한다.

실행 로그 확인

터미널과 `build/sim/run-.../simulation.log`에서 이번 실행을 확인한다.

`LAB1_PASS sub_4bit cases=256`

정상 종료 시각: 2560ns. PASS 문구와 검사 수를 함께 확인한다.

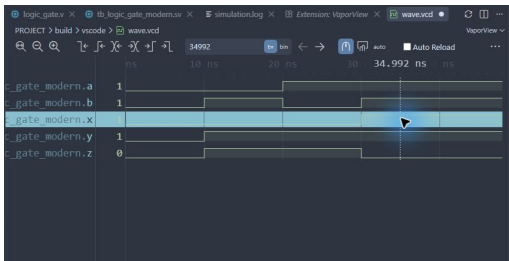
실패하면 이번 실행 폴더의 `compile.log`와 `simulation.log`에서 첫 오류를 찾는다.

코드 저장 → 02 Simulate 재실행 → 새 로그 확인 순서로 복귀한다.
프로젝트 검증 안내

VCD를 파형으로 열기

1. 03 Open waveform 또는 build/sim/wave.vcd를 연다.
2. 텍스트로 열리면 탭 우클릭 → Reopen Editor With... → VaporView.
3. Netlist View에서 tb_sub_4bit을 펼친다.
4. 신호 a[3:0], b[3:0], d[3:0], bor를 각각 두 번 눌러 추가한다.
5. Zoom to Fit를 누르고 Time Units를 ns로 맞춘다.
6. 전환 경계 대신 구간 중간에 커서를 두고 값을 읽는다.

파형 화면의 읽는 순서



위는 공통 파형 조작 예다. 이번 회로에서는 앞 장의 신호 이름을 선택한다.

10~20ns의 0-1과 170~180ns의 1-1에서 borrow가 다름을 확인한다.

신호 이름 → 시간 구간 → 커서 값 → 예상 결과 순서로 비교한다.

실제 VCD의 구간 중간값

시간(ns)	a[3:0]	b[3:0]	d[3:0]	bor
5	0000	0000	0000	0
15	0000	0001	1111	1
35	0000	0011	1101	1
1285	1000	0000	1000	0
2555	1111	1111	0000	0

이 표는 실행된 VCD에서 읽은 값이다. 다중 비트는 이진수다.

표의 시간으로 파형 커서를 옮겨 직접 대조하고, 추가 경계 조건도 확인한다.

constraints에 XDC 직접 작성

1. constraints를 펼치고 pins.xdc 우클릭 → Rename.
2. 파일명: sub_4bit.xdc
3. 핀 표와 코드 이미지를 보며 PACKAGE_PIN·IOSTANDARD·get_ports를 입력한다.
4. get_ports의 이름과 비트 번호를 자신이 작성한 RTL의 포트와 대조한다.
5. File → Save All. Vivado 또는 CLI 구현에는 이 파일을 직접 가져간다.

XDC는 Icarus 기능 시뮬레이션에서 사용하지 않는다. 파형 통과만으로 핀 배치까지 검증됐다고 판단하지 않는다.

XDC 전체 코드 · 1-13행

constraints/sub_4bit.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
1  # Derived from vendor archive; see legacy provenance.
2  set_property PACKAGE_PIN L4 [get_ports bor]
3  set_property PACKAGE_PIN Y1 [get_ports {a[3]}]
4  set_property PACKAGE_PIN W3 [get_ports {a[2]}]
5  set_property PACKAGE_PIN U2 [get_ports {a[1]}]
6  set_property PACKAGE_PIN T1 [get_ports {a[0]}]
7  set_property PACKAGE_PIN W4 [get_ports {b[3]}]
8  set_property PACKAGE_PIN W1 [get_ports {b[2]}]
9  set_property PACKAGE_PIN U4 [get_ports {b[0]}]
10 set_property PACKAGE_PIN M4 [get_ports {d[3]}]
11 set_property PACKAGE_PIN N7 [get_ports {d[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports bor]
13 set_property IOSTANDARD LVCMOS33 [get_ports {a[3]}]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

XDC 전체 코드 · 14-28행

constraints/sub_4bit.xdc · 실제 VS Code 화면을 보며 직접 작성한다.

```
14 set_property IOSTANDARD LVCMOS33 [get_ports {a[2]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports {a[1]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {a[0]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports {b[3]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {b[2]}]
19 set_property IOSTANDARD LVCMOS33 [get_ports {b[1]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {b[0]}]
21 set_property IOSTANDARD LVCMOS33 [get_ports {d[3]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {d[2]}]
23 set_property IOSTANDARD LVCMOS33 [get_ports {d[1]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {d[0]}]
25
26 set_property PACKAGE_PIN V4 [get_ports {b[1]}]
27 set_property PACKAGE_PIN M2 [get_ports {d[2]}]
28 set_property PACKAGE_PIN M7 [get_ports {d[0]}]
```

핀 이름·포트 이름·대괄호·중괄호를 확인하고 저장한다.

코드 수정 실험 · 실패와 복구

1. 정상 PASS 로그를 보관한다. Explorer → src → sub_4bit.v을 연다.
2. borrow 비교에 등호를 추가한다. 테스트벤치의 기대값은 그대로 둔다.
3. File → Save All → Terminal → Run Task... → 02 Simulate.
4. $a=b=0$ 이면 정상 borrow는 0이다. 변경 후에는 1이 된다.
5. 이번 실행의 실패 로그에서 입력·기대값·실제값을 읽는다. 실행 폴더를 기록한다.
6. 바꾼 부분을 원래대로 복구하고 Save All → 02 Simulate. 전체 PASS 와 새 파형을 확인한다.

사전 레포트에 변경 이유·실패 원인·복구 결과를 비교한다. 복구 PASS 뒤에 구현으로 진행한다.

제작 검증: 정상 → 실패 검출 → 복구

실험 전 레포트

1. 목적·포트·비트 폭·예상표와 동작 원리를 설명한다.
 2. 소스와 TB 링크, 코드 커밋, 도구 버전을 남긴다.
 3. 입력 조합·기대값·자동 비교·종료 조건을 설명한다.
 4. 자신의 PASS 로그와 파형 원본·캡처를 첨부한다.
 5. 시간 구간별 값을 해석하고 예상값과 비교한다.
 6. 오류·수정·재실행, 보드에서 확인할 입력과 출력을 적는다.
- 교재 캡처를 자신의 실행 결과로 제출하지 않는다.

Vivado GUI에서 이어서 실습

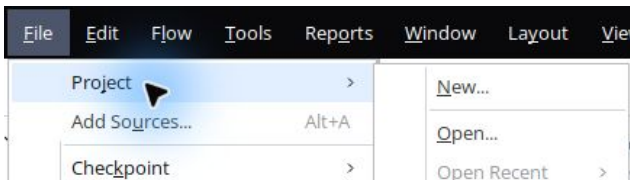
사전 시뮬레이션과 실험 전 레포트를 마친 뒤 Vivado 2026.1을 연다.

이후 단계는 메뉴와 버튼을 직접 누른다.

공통 클릭 화면은 첫 회로에서 실제 캡처했다. 이번 회로의 입력 파일과 top은 다음 슬라이드의 값을 따른다.

첫 회로의 상세 GUI 매뉴얼

File → Project → New...



New Project 안내에서 Next. Project name: sub_4bit

Project location: 자신의 clone 폴더 / vivado

Create project subdirectory를 해제하고 Next. RTL Project와 Do not specify sources at this time을 선택한다.

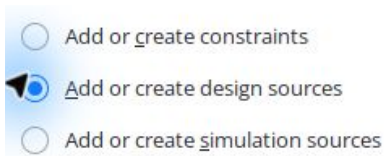
정확한 부품 선택

Parts Boards		Q		
Q: xc7s75fgga484-1		(3 matches)		
Part	I/O Pin Count	Available IOBs	LUT Elements	F
xc7s75fgga484-1	484	338	48000	9
xc7s75fgga484-1IL	484	338	48000	9
xc7s75fgga484-1Q	484	338	48000	9

Parts에서 xc7s75fgga484-1 검색 → 정확히 같은 행 선택 → Next.

요약에서 Spartan-7, fgga484, -1을 확인하고 Finish.

RTL을 Design Sources에 추가



왼쪽 Add Sources → Add or create design sources → Next → Add Files.

등록할 RTL: sub_4bit.v

Copy sources into project를 해제하고 Finish. VS Code와 같은 원본을 참조한다.

Simulation Sources에 TB 추가

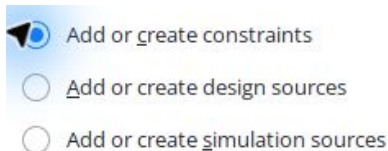
- ☐ Add or create constraints
- ☐ Add or create design sources
- ☒  Add or create simulation sources

Add Sources → Add or create simulation sources → Next → Add Files.

tb_sub_4bit.sv를 선택한다. sim_1을 유지한다.

Copy sources into project는 해제, Include all design sources for simulation은 체크 → Finish.

XDC를 Constraints에 추가



Add Sources → Add or create constraints → Next → Add Files.

sub_4bit.xdc 선택 → Copy constraints files into project 해제 → Finish.

소스 중복 등록이나 잘못된 종류로 추가한 파일이 없는지 확인한다.

두 top을 구분한다

Design Sources의 top: sub_4bit

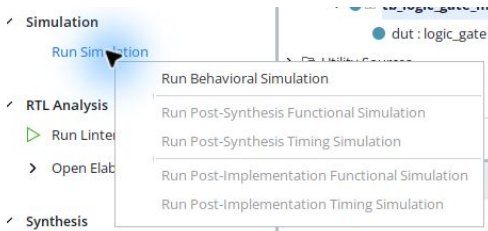
Simulation Sources → sim_1의 top: tb_sub_4bit

Sources의 화살표를 눌러 계층을 펼친다. 굵은 이름이 top이다.

잘못 지정되었다면 올바른 모듈 우클릭 → Set as Top. 이미 top이면 해당 메뉴가 비활성인 것이 정상이다.

테스트벤치를 합성할 설계 top으로 지정하지 않는다.

Run Behavioral Simulation



Run Simulation → Run Behavioral Simulation. 컴파일과 elaboration을 기다린다.

파형에서 신호를 선택하고 Zoom Fit, 시간 단위, 커서 값을 확인한다.

Tcl Console의 PASS 문구와 종료 시각을 확인한다. 오류면 Messages의 첫 오류로 돌아간다.

VS Code 결과와 비교

두 실행은 같은 RTL과 자기검사 TB를 사용한다.

Vivado 실행 상태와 근거 로그는 프로젝트 검증표에서 확인한다.

VS Code 로그: build/sim/run-.../. GUI 로그:
vivado/sub_4bit.sim/sim_1/behav/xsim/.

10-20ns의 0-1과 170-180ns의 1-1에서 borrow가 다를 것을 확인한다.

로그·파형을 서로 다른 이름으로 보관하고 네 가지 정보인 입력·출력·
시간·검사 수를 비교한다.

대상 부품·핀 제약 (1)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
bor	L4
a[3]	Y1
a[2]	W3
a[1]	U2
a[0]	T1
b[3]	W4

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (2)

xc7s75fgga484-1 · IOSTANDARD=LVCMOS33

포트	PACKAGE_PIN
b[2]	W1
b[0]	U4
d[3]	M4
d[1]	N7
b[1]	V4
d[2]	M2

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

대상 부품·핀 제약 (3)

xc7s75fgga484-1 · IOSTANDARD=LVC MOS33

포트	PACKAGE_PIN
d[0]	M7

XDC에서 포트·핀 번호를 대조한다. 다른 보드에는 그대로 쓰지 않는다.

Run Synthesis

- ✓ Synthesis
 - ▶ Run Synthesis
 - > Open Synthesized Design

File → Close Simulation → OK. Flow Navigator → Run Synthesis.

Launch Runs: 기본 디렉터리, Launch runs on local host, PC 자원에 맞는 jobs → OK.

Synthesis successfully completed를 확인한다. 실패했으면 다음 단계로 넘어가지 않는다.

Run Implementation

 Synthesis successfully completed.

Next

- ☒ Run Implementation
- ☐ Open Synthesized Design
- ☐ View Reports

합성 완료 창에서 Run Implementation → OK. Launch Runs에서 OK.
완료 창을 닫았다면 Flow Navigator → Run Implementation.
Implementation successfully completed가 나올 때까지 기다린다.

Generate Bitstream

 Implementation successfully completed.

Next

☐ Open Implemented Design

☒ Generate Bitstream

☐ View Reports

구현 완료 창에서 Generate Bitstream → OK → Launch Runs의 OK.

또는 Program and Debug → Generate Bitstream.

최종 상태 write_bitstream Complete!를 확인한다. 파일 생성과 보드 기록은 다른 단계다.

생성 파일과 보고서

생성 bit: `vivado/sub_4bit.runs/impl_1/sub_4bit.bit`

제작 실행 결과와 증빙은 프로젝트의 `evidence/validation.json`과 README
검증표를 확인한다.

DRC 오류를 확인하고 CFGBVS / CONFIG_VOLTAGE 경고는 실제 보드 구성
전압과 대조한다.

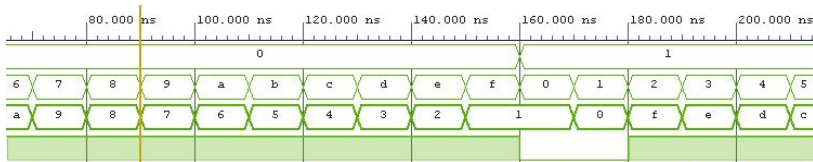
클록 없는 조합회로의 setup/hold NA를 타이밍 검증 완료로 해석하지
않는다.

사용한 커밋·bit 해시·DRC·타이밍 보고서를 결과와 함께 남긴다.

이 회로의 실제 GUI 파형

Untitled 파형 탭 → 오른쪽 위 사각형으로 패널 확대 → Zoom Fit.

신호가 촘촘하면 관심 시간의 파형을 클릭하고 돋보기 +로 확대한다.



위에서부터 a[3:0], b[3:0], d[3:0], bor 순서다.

170-180ns에서 a=b=1이면 d=0, bor=0이다. 180-190ns의 1-2에서는 d=f, bor=1이다.

전환 경계 대신 구간 중간에 커서를 놓고 사전 VCD와 대조한다.

이 회로의 GUI 실행·구현 확인

Log 탭 → Simulation에서 PASS·검사 수·종료 시각을 확인한다.

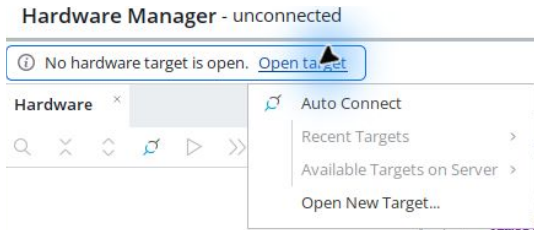
```
Time resolution is 1 ps
LAB1_PASS sub_4bit cases=256
$finish called at time : 2560 ns : File "
```

실제 GUI 실행: 256개 PASS, 종료 2560ns. 정상 종료 후 전체 VCD가 사전 결과와 일치했다.

Name	Constraints	Status
✓ synth_1	constrs_1	synth_design Complete!
✓ impl_1	constrs_1	write_bitstream Complete!

Design Runs의 impl_1: write_bitstream Complete!를 확인한다. 위 구현 화면은 기존 배치 빌드 결과를 GUI에서 연 기록이다.
GUI 실행·전체 파형 비교 근거

실제 보드 연결



Open Hardware Manager → Open target → Auto Connect.

DIP1-4=a[3:0], DIP5-8=b[3:0]. LED1=bor, LED2-5=d[3:0].

장치가 없으면 보드 전원·USB JTAG·드라이버·VM USB 연결을 점검한다.

Program Device와 동작 확인

1. 연결된 장치 이름이 xc7s75인지 확인한다.
2. 장치 우클릭 → Program Device.
3. 이번 프로젝트의 sub_4bit.bit를 선택하고 Program.
4. 기록 완료를 확인한 뒤 입력을 바꾸고 출력과 예상값을 비교한다.
5. 기록 성공 화면과 실제 보드 동작은 각각 증빙한다.

제작 환경의 실제 보드 기록·촬영 검증 여부는 검증표를 따른다. 파일 생성만으로 보드 동작 성공을 선언하지 않는다.

사진과 시연 영상 촬영

DIP1-4=a[3:0], DIP5-8=b[3:0]. LED1=bor, LED2-5=d[3:0].

사진에는 보드 연결과 입력·출력 위치가 함께 보이게 한다.

영상에는 입력을 조작하는 과정과 그에 따른 출력 변화를 담는다.

예상표의 정상·경계 조건을 직접 보여주고 회로 번호를 파일명에 적는다.

통합 영상이라면 각 회로의 타임스탬프를 레포트에서 연결한다.

GitHub 결과 정리

1. reports/pre와 reports/post에 실험 전·후 레포트를 둔다.
 2. evidence/vscode와 evidence/vivado에 로그·파형·캡처를 구분한다.
 3. evidence/board/photos와 videos에 직접 촬영한 자료를 둔다.
 4. 큰 영상·bit는 배포 위치를 정하고 README와 레포트에서 연결한다.
 5. 웹에서 사진 표시와 영상 재생 또는 다운로드가 되는지 확인한다.
- 레포트 양식·예시·GitHub 안내

실험 후 레포트

1. Vivado 버전·part·top·핀 제약·코드 커밋을 기록한다.
2. VS Code와 Vivado의 입력·출력·시간·검사 수를 비교한다.
3. 합성·구현·bit 경로와 경고·수정 사항을 설명한다.
4. 장치 기록 화면·사진·영상과 해당 조건을 연결한다.
5. 예상값·두 시뮬레이션·실측의 일치 또는 차이 원인을 해석한다.
6. 미수행 항목은 미완료로 남기고 후속 확인을 적는다.

전체 목차 PDF 프로젝트로 돌아가기